

Trayecto 2: Sistema inteligente de rutas de distribución mediante árboles y grafos

Karen Nayibe Abril Lozada
Jason Alfredo Castro Navarro

Facultad de Ingeniería y Ciencias Básicas

Universidad Central

CDD-Estructura de Datos_G1_2026-3A

Docente Giovanny Alexandre Briceño Riveros

Bogotá D.C.
2 noviembre de 2026

Introducción

LogiRed Colombia S.A.S. distribuye productos entre varias ciudades del país, pero hoy planea sus rutas a mano, y eso le genera retrasos y errores. La empresa necesita dos cosas distintas: saber cómo está organizada y saber cómo se conectan las ciudades por carretera para escoger por dónde enviar cada entrega. Aunque las dos preguntas tienen que ver con los mismos centros, no tienen la misma forma, y por eso no se resuelven con la misma estructura de datos.

En este proyecto construimos un prototipo en Python con dos estructuras no lineales, un árbol n-ario para la jerarquía organizacional y un grafo no dirigido y ponderado para la red de carreteras. Sobre el árbol se puede insertar, buscar, recorrer y contar; sobre el grafo se pueden agregar conexiones, consultar vecinos, listar todas las rutas posibles entre dos ciudades y calcular la ruta más corta con el algoritmo de Dijkstra.

El documento sigue las fases de la guía: análisis y justificación de las estructuras, diseño del árbol y del grafo, desarrollo del programa, evidencias de las pruebas con Manizales y Santa Marta, análisis de resultados y al final, nuestra reflexión técnica sobre el proceso y las mejoras que le haríamos al sistema.

Análisis del problema

Qué información va en cada estructura

Lo primero fue separar los datos del caso según la forma de la relación que describen. La jerarquía organizacional es una relación de dependencia: cada centro, excepto Bogotá, tiene exactamente un centro del que depende, y un centro puede tener varios dependientes. No hay ciclos: Medellín depende de Bogotá, pero Bogotá no puede depender de Medellín. Esa es justo la definición de un árbol.

La red logística es diferente. Una ciudad puede conectarse con varias, las conexiones van en los dos sentidos y forman ciclos: por ejemplo, Bogotá – Medellín – Pereira – Cali – Bogotá. Además, cada conexión tiene una distancia. Una relación de "muchos con muchos", con ciclos y con un costo en cada enlace, solo se puede representar bien con un grafo ponderado.

Tabla 1

Asignación de la información del caso a cada estructura

Información del caso	Estructura	Razón
Jerarquía de los 7 centros (Bogotá y sus dependientes)	Árbol n-ario	Cada centro tiene un solo padre y puede tener varios hijos; no hay ciclos.
Conexiones entre ciudades con su distancia	Grafo no dirigido y ponderado	Una ciudad se conecta con varias, hay ciclos y cada vía tiene un peso (km).
Nuevos centros (Manizales, Santa Marta)	Árbol	Se insertan como hijos de un centro existente.
Nuevas vías (Pereira–Manizales, Cartagena–Santa Marta)	Grafo	Se agregan como aristas con su peso.
Ruta más corta entre dos centros	Grafo + Dijkstra	Requiere sumar pesos y comparar caminos alternativos.

Cuadro comparativo

Tabla 2

Comparación entre árbol y grafo para el caso LogiRed

Criterio	Árbol (jerarquía)	Grafo (rutas)
Tipo de relación	Padre → hijos (uno a muchos)	Muchos a muchos
Ciclos	No permite	Sí permite (y son útiles: dan rutas alternas)
Pesos	No se necesitan	Distancia en km en cada arista
Dirección	De la raíz hacia las hojas	No dirigido: una vía sirve en ambos sentidos
Operaciones clave	Insertar, buscar, recorridos, contar	Agregar conexión, vecinos, rutas, Dijkstra
Representación elegida	Nodo con lista de hijos	Lista de adyacencia (diccionario de diccionarios)
Pregunta que responde	¿De quién depende cada centro?	¿Por dónde enviar una entrega con menos km?

Organización del trabajo y roles

La guía plantea equipos de tres; nuestro equipo quedó de dos integrantes, así que repartimos las responsabilidades de la siguiente forma y revisamos juntos cada parte antes de integrarla:

Tabla 3

Roles y aportes de cada integrante

Integrante	Rol	Aportes al código y al documento
Jason Alfredo Castro Navarro	Árbol jerárquico, integración y documentación	Clases NodoCentro y ArbolCentros (insertar, buscar, recorridos, contar), menú interactivo, diagramas y redacción del documento.
Karen Nayibe Abril Lozada	Grafo, algoritmos de rutas y pruebas	Clase GrafoRutas (conexiones, vecinos, rutas con DFS y Dijkstra), modo de pruebas de la Fase 4, capturas y análisis de resultados.

Repositorio del proyecto: https://github.com/jcastron4-byte/logired-arboles-grafos

Justificación de las estructuras

Por qué un árbol n-ario y no un árbol binario

Bogotá tiene tres dependientes directos (Medellín, Cali y Barranquilla), así que un árbol binario, que solo admite dos hijos por nodo, no sirve sin forzar el modelo. Por eso cada nodo guarda

una lista de hijos de tamaño variable. Tampoco usamos un árbol binario de búsqueda: los centros no tienen un orden natural que acelere la búsqueda, porque lo que importa es la dependencia y no el orden alfabético o el código.

Con esta estructura, insertar un centro bajo un padre conocido es inmediato una vez se encuentra el padre, y los recorridos son naturales: el preorden muestra la cadena de mando de arriba hacia abajo, el postorden sirve para consolidar información de las hojas hacia la sede principal (por ejemplo, sumar inventarios) y el recorrido en anchura muestra los niveles de la organización. La búsqueda cuesta O(n) porque el árbol no está ordenado; con 9 centros es insignificante, y en la reflexión proponemos cómo bajarla a O(1).

Por qué un grafo ponderado con lista de adyacencia

Las vías tienen distancia, así que el grafo debe ser ponderado; y como una carretera se recorre en ambos sentidos, es no dirigido: cada conexión se guarda dos veces (A→B y B→A). Para representarlo escogimos una lista de adyacencia con diccionarios, {ciudad: {vecino: km}}, en lugar de una matriz de adyacencia. La red es dispersa: con 9 ciudades hay 11 vías, mientras que una matriz tendría 81 casillas, la mayoría vacías. La lista ocupa O(V + E) memoria, consultar los vecinos de una ciudad es directo y agregar una vía cuesta O(1).

Por qué Dijkstra para la ruta más corta

Un recorrido en anchura (BFS) encuentra la ruta con menos tramos, pero no la de menos kilómetros. En nuestro caso eso importa: de Bogotá a Cartagena la ruta con menos tramos pasa por Medellín (2 tramos, 1058 km), pero la más corta en distancia pasa por Bucaramanga y Barranquilla (3 tramos, 1052 km). Dijkstra sí tiene en cuenta los pesos y garantiza el camino mínimo cuando todas las distancias son positivas, lo que siempre se cumple con kilómetros. Lo implementamos con una cola de prioridad (heapq), así su costo es O((V + E) log V).

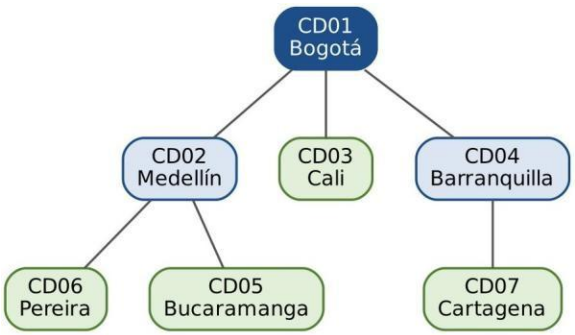
Para la opción de "mostrar rutas disponibles" usamos además una búsqueda en profundidad (DFS) con backtracking que lista todos los caminos simples entre dos ciudades. Esto no reemplaza a Dijkstra, porque su costo crece muy rápido con el tamaño de la red, pero con 9 ciudades sirve para mostrarle al usuario las alternativas y para comprobar que Dijkstra eligió la menor.

Diseño del árbol

El árbol inicial se construyó con los siete centros de la guía (Figura 1). Su raíz es Bogotá (CD01), la sede principal; los nodos internos son Medellín y Barranquilla, que tienen dependientes; y las hojas son Cali, Pereira, Bucaramanga y Cartagena. Tiene 7 nodos y 3 niveles.

Figura 1

Árbol jerárquico inicial de los centros de distribución



Nota. Azul oscuro: raíz; azul claro: nodos internos; verde: hojas.

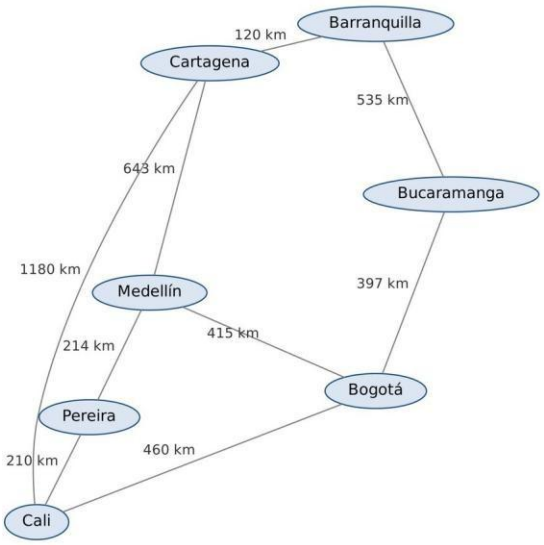
Tabla 4
Recorridos del árbol inicial

Recorrido	Resultado
Preorden	Bogotá → Medellín → Pereira → Bucaramanga → Cali → Barranquilla → Cartagena
Postorden	Pereira → Bucaramanga → Medellín → Cali → Cartagena → Barranquilla → Bogotá
Anchura	Bogotá → Medellín → Cali → Barranquilla → Pereira → Bucaramanga → Cartagena

Diseño del grafo

El grafo inicial tiene 7 vértices (las ciudades) y 9 aristas (las vías), cada una con su distancia en kilómetros (Figura 2). Las posiciones de los vértices se acomodaron de forma aproximada a la geografía del país para que el diagrama se lea con facilidad. Los vértices de mayor grado son Bogotá, Medellín, Cali y Cartagena, con tres conexiones cada uno.

Figura 2
Grafo inicial de la red logística con distancias en km



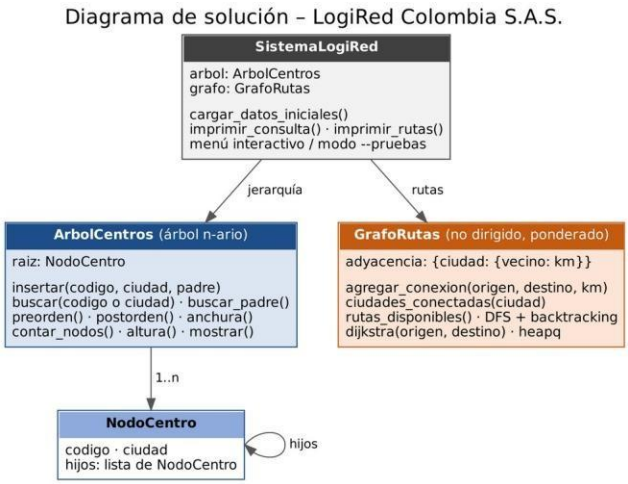
Al comparar las Figuras 1 y 2 se ve por qué hacían falta dos estructuras: la jerarquía no coincide con las vías. Bucaramanga depende de Medellín en la organización, pero no tiene ninguna carretera directa hacia Medellín; se conecta con Bogotá y con Barranquilla. Si se usara el árbol para planear rutas, se buscarían caminos que no existen.

Desarrollo del programa

El prototipo está en un solo archivo, `logired.py`, con cuatro clases (Figura 3). Se ejecuta con `python logired.py` para usar el menú interactivo, o con `python logired.py --pruebas` para correr automáticamente las pruebas de la Fase 4. El código completo y comentado va en el archivo .zip; aquí mostramos los fragmentos más importantes.

Figura 3

Diagrama de solución: clases del prototipo y sus relaciones



Árbol: insertar un centro bajo su padre

```
def insertar(self, codigo, ciudad, codigo_padre=None):
    if self.buscar(codigo) is not None:           # evita códigos repetidos
        return False
    nuevo = NodoCentro(codigo, ciudad)
    if self.raiz is None:                         # primer centro -> raíz
        self.raiz = nuevo
        return True
    padre = self.buscar(codigo_padre)
    if padre is None:                             # el padre debe existir
        return False
    padre.hijos.append(nuevo)                     # se enlaza como dependiente
    return True
```

Árbol: recorridos en preorden y en anchura

```
def preorden(self):
    resultado = []
    def visitar(nodo):
        resultado.append(nodo.ciudad)    # 1. primero el nodo
        for hijo in nodo.hijos:          # 2. luego cada subárbol
            visitar(hijo)
    visitar(self.raiz)
    return resultado

def anchura(self):                       # nivel por nivel con una cola FIFO
    cola, resultado = deque([self.raiz]), []
    while cola:
        actual = cola.popleft()
        resultado.append(actual.ciudad)
        cola.extend(actual.hijos)
    return resultado
```

Grafo: algoritmo de Dijkstra con cola de prioridad

```
def dijkstra(self, origen, destino):
    distancia = {c: float("inf") for c in self.adyacencia}
    anterior  = {c: None for c in self.adyacencia}
    distancia[origen] = 0
    cola, visitados = [(0, origen)], set()
    while cola:
        dist_actual, actual = heapq.heappop(cola)    # la más cercana
        if actual in visitados: continue
        visitados.add(actual)
        if actual == destino: break
        for vecino, peso in self.adyacencia[actual].items():
            nueva = dist_actual + peso
            if nueva < distancia[vecino]:              # relajación
                distancia[vecino] = nueva
                anterior[vecino] = actual
                heapq.heappush(cola, (nueva, vecino))
    camino, paso = [], destino                         # reconstrucción
    while paso is not None:
        camino.append(paso); paso = anterior[paso]
    return distancia[destino], camino[::-1]
```

A diferencia del ejemplo del material del curso, que solo devuelve las distancias, nuestra versión guarda en anterior desde qué ciudad se llegó a cada vértice. Así, al terminar, se puede reconstruir el camino completo de atrás hacia adelante y mostrarle al usuario por qué ciudades pasar, que es lo que necesita la empresa.

Evidencias de ejecución

Las pruebas se ejecutaron con Python 3 usando el comando python logired.py --pruebas, y luego se probó el menú interactivo. Las capturas muestran la salida real del programa.

Figura 4

Estado inicial: árbol y lista de adyacencia cargados con los datos de la guía

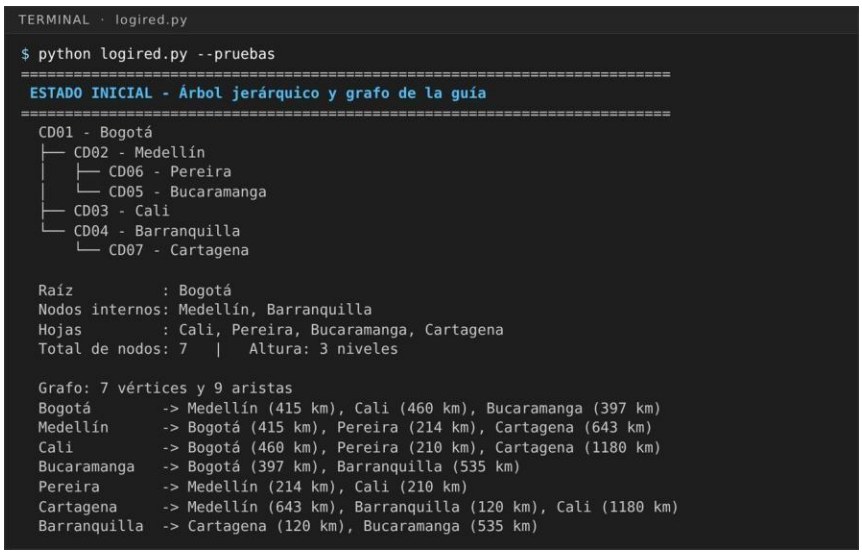


Figura 5

Prueba 1: inserción de Manizales y Santa Marta, validaciones y árbol actualizado

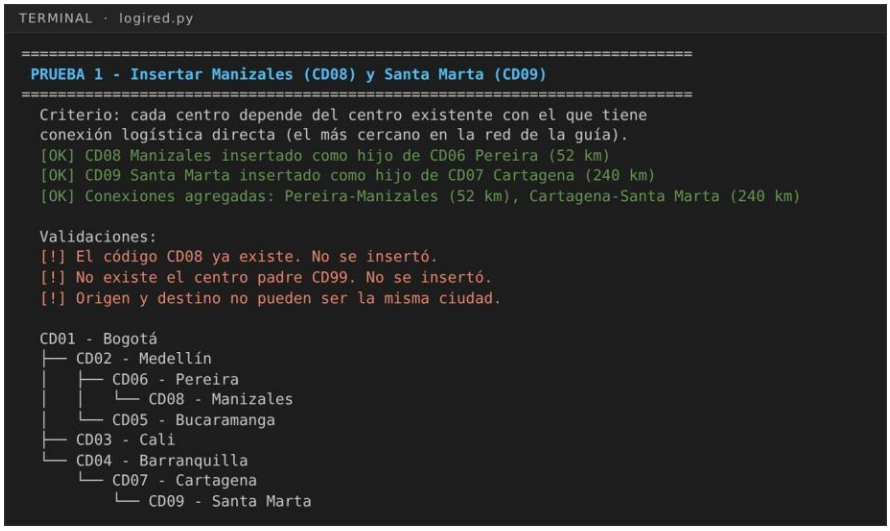


Figura 6

Prueba 2: consulta de centros por código y por nombre

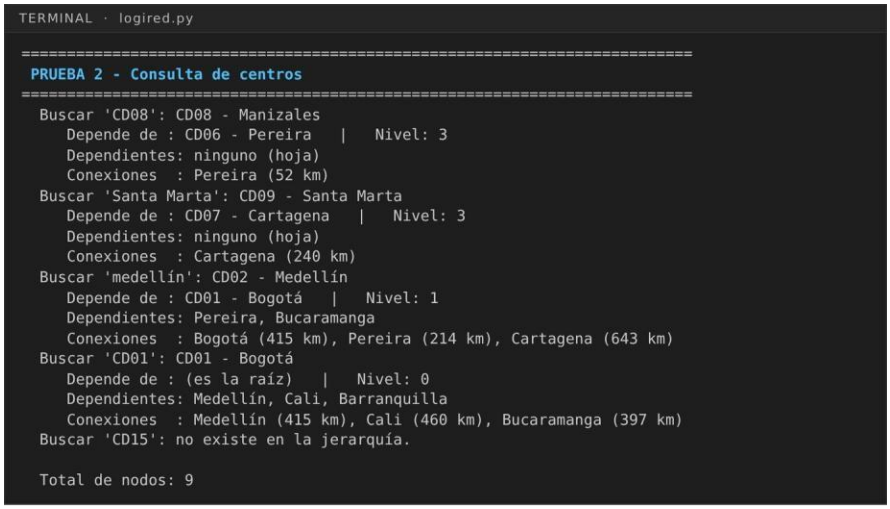


Figura 7

Prueba 3: recorridos del árbol actualizado

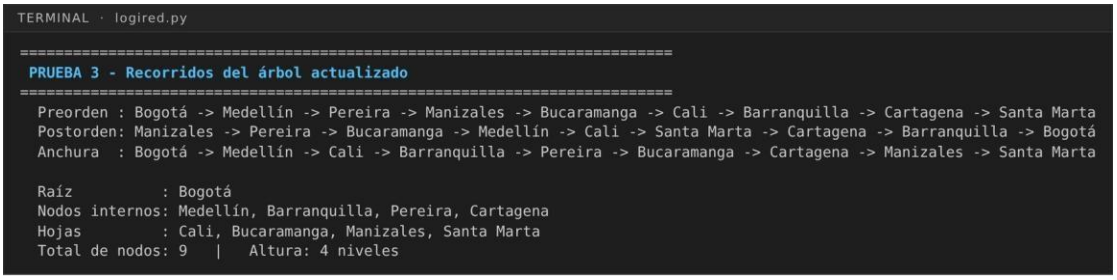


Figura 8

Prueba 4: grafo actualizado con 9 vértices y 11 aristas

```
TERMINAL · logired.py

================================================================================
 PRUEBA 4 - Grafo actualizado (lista de adyacencia)
================================================================================
  9 vértices y 11 aristas

  Bogotá        -> Medellín (415 km), Cali (460 km), Bucaramanga (397 km)
  Medellín      -> Bogotá (415 km), Pereira (214 km), Cartagena (643 km)
  Cali          -> Bogotá (460 km), Pereira (210 km), Cartagena (1180 km)
  Bucaramanga   -> Bogotá (397 km), Barranquilla (535 km)
  Pereira       -> Medellín (214 km), Cali (210 km), Manizales (52 km)
  Cartagena     -> Medellín (643 km), Barranquilla (120 km), Cali (1180 km), Santa Marta (240 km)
  Barranquilla  -> Cartagena (120 km), Bucaramanga (535 km)
  Manizales     -> Pereira (52 km)
  Santa Marta   -> Cartagena (240 km)
```

Figura 9

Prueba 5: análisis de rutas Bogotá–Cartagena y Bogotá–Manizales

```
TERMINAL · logired.py

================================================================================
 PRUEBA 5 - Análisis de rutas con Dijkstra
================================================================================
  Rutas disponibles Bogotá -> Cartagena: 5
     1. Bogotá -> Bucaramanga -> Barranquilla -> Cartagena  = 1052 km
     2. Bogotá -> Medellín -> Cartagena  = 1058 km
     3. Bogotá -> Cali -> Pereira -> Medellín -> Cartagena  = 1527 km
     4. Bogotá -> Cali -> Cartagena  = 1640 km
    ... y 1 ruta más larga
  Dijkstra (menor distancia): Bogotá -> Bucaramanga -> Barranquilla -> Cartagena = 1052 km

  Rutas disponibles Bogotá -> Manizales: 6
     1. Bogotá -> Medellín -> Pereira -> Manizales  = 681 km
     2. Bogotá -> Cali -> Pereira -> Manizales  = 722 km
     3. Bogotá -> Bucaramanga -> Barranquilla -> Cartagena -> Medellín -> Pereira -> Manizales  = 1961 km
     4. Bogotá -> Bucaramanga -> Barranquilla -> Cartagena -> Cali -> Pereira -> Manizales  = 2494 km
    ... y 2 rutas más largas
  Dijkstra (menor distancia): Bogotá -> Medellín -> Pereira -> Manizales = 681 km
```

Figura 10

Prueba 5: análisis de rutas Cali–Barranquilla y Bogotá–Santa Marta

```
TERMINAL · logired.py

  Rutas disponibles Cali -> Barranquilla: 6
     1. Cali -> Pereira -> Medellín -> Cartagena -> Barranquilla  = 1187 km
     2. Cali -> Cartagena -> Barranquilla  = 1300 km
     3. Cali -> Bogotá -> Bucaramanga -> Barranquilla  = 1392 km
     4. Cali -> Bogotá -> Medellín -> Cartagena -> Barranquilla  = 1638 km
    ... y 2 rutas más largas
  Dijkstra (menor distancia): Cali -> Pereira -> Medellín -> Cartagena -> Barranquilla = 1187 km

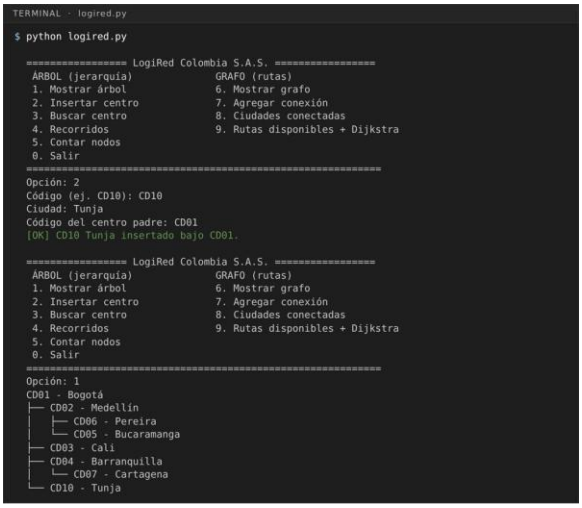
  Rutas disponibles Bogotá -> Santa Marta: 5
     1. Bogotá -> Bucaramanga -> Barranquilla -> Cartagena -> Santa Marta  = 1292 km
     2. Bogotá -> Medellín -> Cartagena -> Santa Marta  = 1298 km
     3. Bogotá -> Cali -> Pereira -> Medellín -> Cartagena -> Santa Marta  = 1767 km
     4. Bogotá -> Cali -> Cartagena -> Santa Marta  = 1880 km
    ... y 1 ruta más larga
  Dijkstra (menor distancia): Bogotá -> Bucaramanga -> Barranquilla -> Cartagena -> Santa Marta = 1292 km
```

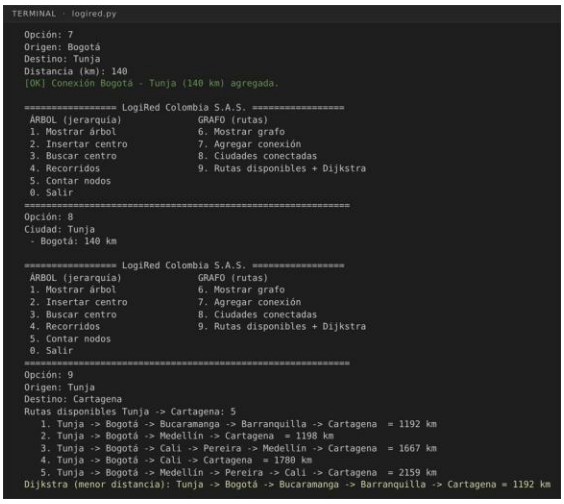
Figura 11

Menú: inserción de Tunja bajo Bogotá

Figura 12

Menú: nueva conexión, vecinos y Dijkstra desde Tunja





Resultados obtenidos

Inserción de los nuevos centros y justificación del nodo padre

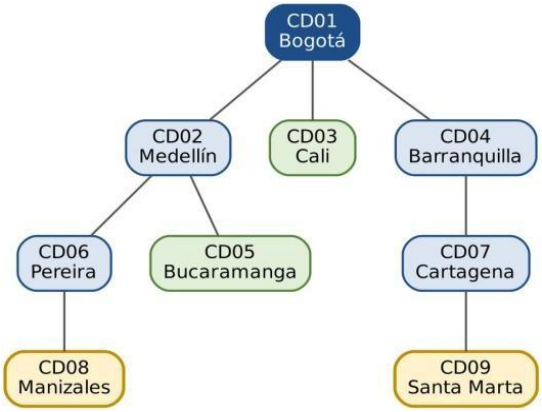
Para decidir de quién depende cada centro nuevo usamos un mismo criterio: el padre es el centro existente con el que el nuevo tiene conexión logística directa, porque será el que lo abastezca y supervise en la operación diaria.

- Manizales (CD08) → Pereira (CD06). Su única vía en la red es con Pereira, a 52 km, la conexión más corta de todo el sistema. Pereira ya pertenece a la región de Medellín, así que Manizales queda en el Eje Cafetero bajo esa misma línea de mando.
- Santa Marta (CD09) → Cartagena (CD07). Su única vía es con Cartagena (240 km). Así queda en la región Caribe, que coordina Barranquilla.

Otra opción era colgar ambos centros directamente de los jefes regionales (Medellín y Barranquilla). La descartamos porque esos centros no tienen vía directa con los nuevos, y la dependencia quedaría solo en el papel. Con nuestro criterio, Pereira y Cartagena pasan de ser hojas a nodos internos y el árbol crece de 3 a 4 niveles (Figura 13).

Figura 13

Árbol actualizado con Manizales y Santa Marta



Nota. En amarillo, los centros insertados en las pruebas.

Consultas, recorridos y conteo

La búsqueda funcionó por código (CD08) y por nombre, sin importar mayúsculas ("medellín"), y para cada centro mostró su padre, su nivel, sus dependientes y sus conexiones. La búsqueda de un código inexistente (CD15) se manejó sin errores. El conteo pasó de 7 a 9 nodos. Los recorridos del árbol actualizado fueron:

Tabla 5

Recorridos del árbol actualizado (9 nodos, 4 niveles)

Recorrido	Resultado
Preorden	Bogotá → Medellín → Pereira → Manizales → Bucaramanga → Cali → Barranquilla → Cartagena → Santa Marta
Postorden	Manizales → Pereira → Bucaramanga → Medellín → Cali → Santa Marta → Cartagena → Barranquilla → Bogotá
Anchura	Bogotá → Medellín → Cali → Barranquilla → Pereira → Bucaramanga → Cartagena → Manizales → Santa Marta

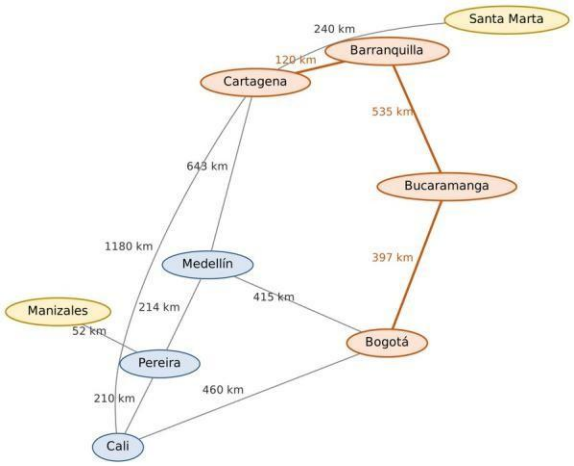
También se probaron las validaciones: el programa rechazó un código repetido (CD08), un padre inexistente (CD99) y una conexión de una ciudad consigo misma, sin detenerse.

Grafo actualizado y análisis de rutas

Con las dos vías nuevas el grafo quedó con 9 vértices y 11 aristas (Figura 14). Para el análisis de rutas comparamos, en cuatro casos, todas las rutas posibles encontradas con DFS contra el resultado de Dijkstra:

Figura 14

Grafo actualizado con la ruta más corta Bogotá–Cartagena resaltada



Nota. En naranja, la ruta encontrada por Dijkstra; en amarillo, las ciudades nuevas.

Tabla 6

Análisis de rutas: alternativas encontradas y resultado de Dijkstra

Origen → destino	Rutas	Ruta más corta (Dijkstra)	Segunda opción
Bogotá → Cartagena	5	Bogotá → Bucaramanga → Barranquilla → Cartagena = 1052 km	Por Medellín: 1058 km

Bogotá → Manizales	6	Bogotá → Medellín → Pereira → Manizales = 681 km	Por Cali: 722 km
Cali → Barranquilla	6	Cali → Pereira → Medellín → Cartagena → Barranquilla = 1187 km	Directo por Cartagena: 1300 km
Bogotá → Santa Marta	5	Bogotá → Bucaramanga → Barranquilla → Cartagena → Santa Marta = 1292 km	Por Medellín: 1298 km

En los cuatro casos Dijkstra eligió la misma ruta que quedó primera en la lista de todas las alternativas, lo que confirma que el algoritmo funciona. Los resultados también muestran por qué no conviene planear a mano: de Bogotá a Cartagena la ruta "lógica" por Medellín tiene menos tramos, pero la más corta va por Bucaramanga y Barranquilla, aunque solo por 6 km. De Cali a Barranquilla pasa algo parecido: la vía directa Cali–Cartagena (1180 km) parece obvia, pero dar la vuelta por Pereira y Medellín ahorra 113 km.

Reflexión técnica

Por qué el árbol fue la estructura adecuada para la jerarquía

La jerarquía de LogiRed cumple las tres condiciones de un árbol: una sola raíz, un único padre por centro y ningún ciclo. El árbol hizo que esas reglas quedaran en la estructura misma: no hay forma de que un centro dependa de dos jefes a la vez. Además, cada recorrido respondió una pregunta real de la empresa: el preorden da la cadena de mando, la anchura muestra quién está en insertar Manizales y Santa Marta: bastó con enlazarlos a su padre y los recorridos, el conteo y la altura se actualizaron solos.

Por qué el grafo fue la estructura adecuada para las rutas

La red de carreteras no tiene jerarquía: tiene ciclos, conexiones en ambos sentidos y una distancia en cada tramo. Justamente los ciclos son los que crean rutas alternativas, y sin ellos no habría nada que optimizar. El grafo ponderado permitió aplicar Dijkstra y encontrar resultados que no eran evidentes a simple vista, como el caso Bogotá–Cartagena. Intentar usar el árbol para esto habría sido un error, porque, como vimos con Bucaramanga, la dependencia administrativa no coincide con las carreteras.

Ventajas y limitaciones encontradas

- **Ventaja – separación de responsabilidades.** Árbol y grafo son independientes: cambiar una vía no altera la jerarquía, y reorganizar la empresa no rompe las rutas. Las dos estructuras se conectan solo por el nombre de la ciudad.
- **Ventaja – estructuras dinámicas.** No hubo que definir un tamaño máximo: los centros y las vías nuevas se agregaron en tiempo de ejecución, tanto en las pruebas como desde el menú (Tunja).
- **Limitación – búsqueda lineal en el árbol.** Buscar un centro o su padre exige recorrer el árbol (O(n))Además, `nivel()` llama varias veces a `buscar_padre()`, lo que en el peor caso lo vuelve O(n^2). Con 9 nodos no se nota, pero con cientos de centros sí.
- **Limitación – DFS exponencial.** Listar todas las rutas simples crece de forma exponencial con el tamaño de la red. Con 9 ciudades hay entre 5 y 6 rutas por par, pero con una red nacional real esta opción se volvería impráctica.
- **Limitación – sin persistencia y sin eliminación.** Los datos se cargan desde el código y se pierden al cerrar el programa; tampoco implementamos eliminar centros o vías, que

obligaría a decidir qué pasa con los dependientes de un centro borrado.

Mejoras futuras

- **Índice con diccionario para búsquedas O(1).** Mantener un diccionario {codigo: nodo} que se actualice al insertar. Buscar un centro pasaría de O(n) a O(1), y si cada nodo guarda además una referencia a su padre, nivel() pasaría de O(n^2) a O(h), donde h es la altura del árbol.
- **Precalcular o guardar en caché las rutas más cortas.** Las consultas de rutas se repiten mucho en logística. Se puede guardar el resultado de Dijkstra por origen y borrarlo solo cuando cambie una vía, o precalcular todas las parejas con Floyd–Warshall (O(V3)) cuando la red cambia poco.
- **Algoritmo A estrella con coordenadas.** Si cada ciudad guarda su latitud y longitud, el algoritmo A* usa la distancia en línea recta como guía y explora muchos menos vértices que Dijkstra en redes grandes, con el mismo resultado óptimo.
- **Limitar la búsqueda de alternativas.** Cambiar el DFS completo por el algoritmo de Yen, que obtiene solo las k rutas más cortas, para que la opción de rutas disponibles siga siendo útil cuando la red crezca.
- **Persistencia y pesos reales.** Guardar árbol y grafo en JSON o en una base de datos, y usar como peso el tiempo o el costo del viaje (peajes, tráfico) en lugar de solo la distancia.

Conclusiones

1) El caso de LogiRed tenía dos tipos de información con formas distintas: una jerarquía sin ciclos y una red con ciclos y pesos. Elegir una estructura distinta para cada una fue la decisión más importante del proyecto.
2) El árbol n-ario, con lista de hijos, representó la jerarquía de forma natural, permitió insertar Manizales y Santa Marta con un criterio justificado y ofreció tres recorridos con un significado claro para la empresa.
3) El grafo no dirigido y ponderado, guardado como lista de adyacencia, modeló la red de forma eficiente, y Dijkstra encontró en todos los casos la ruta de menor distancia, coincidiendo con la comparación exhaustiva de todas las rutas posibles.
4) Las pruebas mostraron que la ruta más corta no siempre es la más evidente, lo que justifica automatizar la planeación que hoy LogiRed hace a mano.
5) El prototipo es funcional, pero para una operación real conviene agregar índices de búsqueda, caché de rutas y persistencia de datos, como planteamos en la reflexión.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Joyanes Aguilar, L. y Zahonero Martínez, I. (2008). *Estructuras de datos en Java*. McGraw-Hill.

Python Software Foundation. (s. f.). *heapq — Heap queue algorithm*. Python 3 documentation. https://docs.python.org/3/library/heapq.html

Python Software Foundation. (s. f.). *Data structures*. Python 3 documentationhttps://docs.python.org/3/tutorial/datastructures.html

Universidad Central. (2026). *Estructuras de datos no lineales* [Material de estudio]. UCentral Virtual.

Universidad Central. (2026). *Guía de actividad: Sistema inteligente de rutas de distribución mediante árboles y grafos*. UCentral Virtual.